

Problems on Strings

① Count number of vowels in a string

s = "Hello how are you"

vow = "aeiouAEIOU"

for i in s:

s[0] → H
s[1] → e

```
1 def countVowels(s):  
2     count = 0  
3     vow = 'aeiouAEIOU'  
4     for i in s:  
5         if i in vow:  
6             count+=1  
7     return count  
8  
9 res = countVowels("hello world how are yoy")  
10 print(res)
```

② Reverse String:

s = "hello"

res → "olleh"

0 1 2 3 4

h e l l o

↑

len(s) → n

range(n-1, -1, -1)
↑

res = "" + 'o' → 'o' ← 4
res = "o" + 'l' → 'ol' ← 3
res = "ol" + 'l' → 'oll' ← 2
res = "oll" + 'e' → 'olle' ← 1
res = "olle" + 'h' → 'olleh' ← 0

③ Convert to upper case (manually)

s = "hello"

a → 97

A → 65

res → "HELLO"

'a' → ord('a') → 97

97 → chr(97) → 'a'

h e l l o
104 101 108 108 111

↓

72 69 76 76 79

H E L L O

- 32 - 32 - 32 - 32 - 32

⑤ Swap case (lower $\rightarrow$ upper)
(upper $\rightarrow$ lower)

Diagram illustrating string reversal:

- Original string: `Hello` (black text)
- Reversed string: `olleH` (red text)
- Original string: `abcd` (red text)
- Reversed string: `dcbA` (blue text)

97 - 122 $\rightarrow$ if ($x \geq 97$ and $x \leq 122$)
-32

else if ($x \geq 65$ and $x \leq 90$)
+32

⑥ Check for palindromes

raceCar = raceCar ✓

① reverse string hello == olleh



0 1 2 3 4 5 6
racecar
↑ ↑ |

$$S[i] = S[n-i-1]$$

1

Panagram check

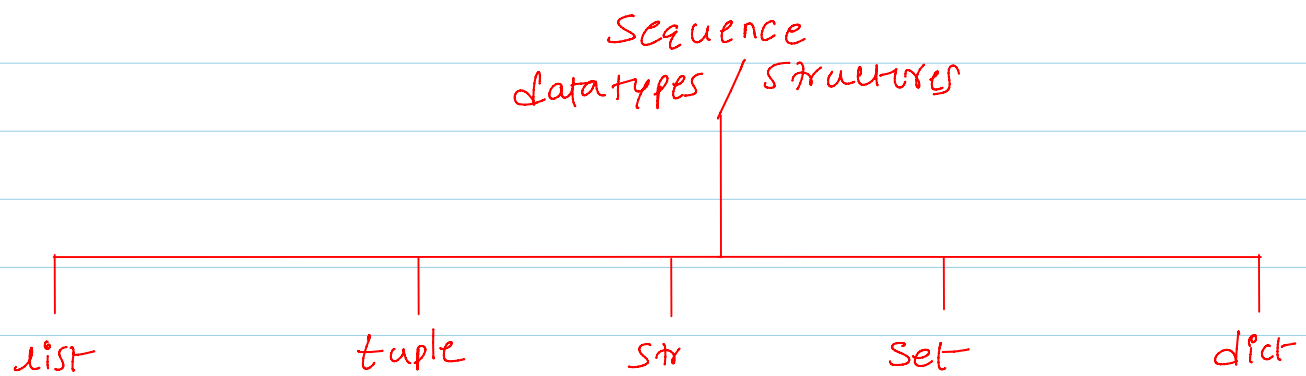
given a string and check if all 26 alphabets
in that string / not

if 26 alphabets are there $\Rightarrow$ panagram
if not $\rightarrow$ Not

"abcdefghijklmnopqrstuvwxyz"

"The quick brown fox jumps over the lazy dog"

Non primitives (collection)



Mutable :- once we created the data structure we can modify / the data structure without re assign

Immutable :- once we created we can't modify the data structure
→ if we want re-assign

1) list :- → mutable, ordered, can accept any type and Dynamic in nature (can grow and shrink)
→ allow duplicates also

Syntaxes:-

arr = [] // default

arr = list() // constructor way

→ as list is mutable we can do

1) Access

2) updation

3) Insertion → append()

4) Deletion → pop / del

1) Access:

1) through indexing

arr = [10, 20, 30, 40]

arr[0] → 10

arr[-1] → 40

0	1	2	3
10	20	30	40
-4	-3	-2	-1

2) Through slicing operator

arr = [10, 20, 30, 40]

arr[0:3] → [10, 20]

arr[-2:-4:-1] → [30, 20]

2) update:

1) Indexing:

arr = [10, 20, 30, 40]

arr[0] = 200

[10, 20, 30, 40]

[200, 20, 30, 40]

2) through slicing operator

arr[0:2] = [300, 400]

[10, 20, 30, 40] [300, 400, 30, 40]

3) deletion

del arr[0] → [20, 30, 40]

Methods

- 1) append(x) → adds data at last
- 2) pop() → deletes the last element
- 3) extend(list) → adds new list elmts
- 4) insert(idx, val) → insert value at idx
- 5) index(value) → return idx of value
- 6) len(arr) → return length of arr
- 7) ~~*~~ arr.sort() → sorts the array

Aggregation functions

min(arr)

max(arr)

sum(arr)

avg(arr)

} only work upon integer arrays

2) Tuple :

- a tuple is immutable,
- can have duplicates
- can store any type of data

Syntax;

t = (,) // general

t = tuple() // constructor

t = 1, 2, 3, //

t = tuple([1, 2, 3])

→ as tuple is immutable we can't -

{
add
modify
delete
} data

→ we can only Access Data

1) indexing

2) slicing

Methods:

1) .index(elem) → return index of element

2) .count(elem) → return count of element

3) Set:

- Set is mutable in nature and it don't allow duplicates,
- it will only allow you to store immutable types in it
- has no index
- it can't maintain insertion order

Immutable types → int, float, bool
tuple, string

Syntax:

`s = set()`

`s = {1, 2, 3, 4}`

→ as Set is mutable we can

- 1) add
- 2) delete
- 3) modify
- 4) access

Method:

1) `add(elem)`

→ add data to set

2) `pop()`

→ delete random data

3) `discard(elem)`

→ delete element

4) `clear()`

→ all elements deleted.

4) Dictionary

→ it is a data structure which will store data in key: value pairs

→ keys are immutable types and values are anything you want

→ it should not allow duplicate keys

Syntax:

`d = {}`

`d = dict()`

```
d = { "name" : "pavan",  
      "age"  : 24  
}
```

Methods:

1) `d[key] = val`

→ If there is key it will update value

→ else add new key: value

2) `d[key]` / `d.get(key)` → if key you will get data else not

→ If no key you will get error

③ d.keys() → return all keys

4) d.values() → return all values

5) d.items() → return items in form of
[(key,value),(key,value)]

6) d.popitem() → delete last inserted (key:value)

7) d.pop(key) → deletes the (key:value)

